Universidad Nacional Autónoma de México

Facultad de Ingeniería

Exposición
Planificación de Tareas y Ejecución Paralela

Sistemas Operativos
Grupo 07

Profesor: Dr. Gunnar Eyal Wolf Iszaevich

Integrantes:
Estrada Zacarias Aldo Axel
Sánchez Salazar Jazmín

Semana de exposición: Abril 13, 2026

Semestre 2026-2

Índice

Introducción **2**

1. Paralelismo de Datos vs Paralelismo de Tareas **2**

2. Grafos de Tareas y Ruta Crítica **3**

3. Granularidad de Tareas **3**

4. Thread pools (pool de hilos) **4**

5. Sobrecosto de Paralelización (Overhead) **5**

6. Ley de Amdahl y Speedup **5**

7. Ley de Gustafson **7**

Conclusión **7**

Bibliografía **8**

Introducción

En los sistemas operativos la planificación de tareas y la ejecución paralela son fundamentales, ya que nos permiten tener una organización de cómo es que se va a distribuir el trabajo entre los procesos, hilos y los recursos computacionales. En general, estos procesos no son visibles para los usuarios finales, por lo cual es necesario implementarlos de forma correcta, ya que impacta directamente en el desempeño, la eficiencia y la capacidad de respuesta del sistema.
A lo largo de este documento se desarrollarán distintos temas relacionados con la forma en la que el sistema operativo de nuestros dispositivos coordina diversas ejecuciones, tomando en cuenta la organización del trabajo, cómo se aprovecha el procesamiento disponible, así como algunos factores que impactan de forma directa al rendimiento en el momento que varias tareas se ejecutan de manera paralela.

1. Paralelismo de Datos vs Paralelismo de Tareas

En la computación paralela buscamos que varios procesos sean realizados al mismo tiempo y para lograr esto hay diversas formas en las que se puede dividir el trabajo y así **aprovechar de mejor manera los recursos** de cómputo que tenemos disponibles, ahí es donde entran estos dos enfoques, el **paralelismo de datos** y el **paralelismo de tareas**. Aunque tengan en común el querer ejecutar varias actividades de manera simultánea, ambas difieren en la forma de dividir los trabajos en forma paralela.

La principal característica del **paralelismo de datos** es que **distribuye de manera uniforme** un conjunto grande de **información en varias partes**, con el fin de que **varios hilos apliquen** en simúltaneo **una misma operación, o proceso**, sobre cada una de esas partes. De este modo cada uno de los hilos realiza el mismo trabajo, pero sobre distintas partes de la información. Este enfoque resulta conveniente cuando se tiene una **estructura de la información repetitiva o parecida** entre sí, ya que reparte la carga de forma equitativa. Para entender de mejor manera este tipo de paralelismo podemos tomar como ejemplo una imagen, la cual se divide en bloques y a varios de esos bloques se les aplica un filtro al mismo tiempo, o una base de datos, de la cual se tienen que realizar un análisis de múltiples registros.

En el caso del **paralelismo de tareas** se divide el problema en actividades distintas, pero más pequeñas, de este modo cada uno de los hilos realiza procesos diferentes entre sí, que ayudan a resolver el problema general. Aquí tenemos a los **hilos colaborando en diferentes etapas** del proceso de ejecución para poder resolver la problemática principal en conjunto, donde cada uno tiene un trabajo específico. Un ejemplo básico para este tipo de paralelismo sería tener una aplicación que recibe datos, un hilo se encarga de recibir los datos, otro de procesarlos y otro que genera algún tipo de salida o los almacena.

Como se puede observar hay una clara diferencia en la forma en que trabaja cada tipo de paralelismo, siendo que uno **divide la información que recibe**, mientras el otro **divide los trabajos que realiza**. Es importante notar esta diferencia ya que entender cómo funcionan estos procesos influye de manera directa en cómo **planificar las actividades e incluso los mecanismos de coordinación a utilizar**. Sin embargo, cada uno de estos tipos de paralelismo **presenta distintos retos**, en el paralelismo de datos es importante **dividir la información de manera adecuada**, para repartir de forma eficiente la carga de trabajo, y una vez obtenidos los resultados, es importante saber cómo **reunir esos resultados parciales**; mientras que en el paralelismo de tareas es importante saber cómo **comunicar los diferentes procesos** y **cómo sincronizar las diferentes etapas** llevadas a cabo en la ejecución.

Podemos **relacionar estos tipos de paralelismos a conceptos y ejemplos vistos en clase**. Por ejemplo el **paralelismo de datos** fácilmente se asocia con el **patrón de equipo de trabajo**, ya que tenemos a **varios hilos que realizan una misma operación** sobre diferentes conjuntos de datos. En el caso del **paralelismo de tareas** se puede relacionar con los **modelos jefe/trabajador** y con el ejemplo de la **línea de ensamblado**, ya que en ellos las **actividades se distribuyen en diferentes funciones**.

2. Grafos de Tareas y Ruta Crítica

Al momento de dividir un problema en varias actividades que se requiere ejecutarlas de manera paralela es importante entender que **no siempre**, o no todas las **tareas se pueden paralelizar**, ya que en muchos casos hay tareas que se deben terminar de forma obligatoria, antes de que otra inicie su ejecución, ya que hay **dependencia entre esas actividades**. Para poder modelar de una manera más clara las relaciones que hay entre las actividades es necesario un **grafo de actividades**, el cual nos permite visualizar la estructura del problema, e **identificar en qué partes el problema se puede paralelizar** y en cuáles se debe respetar un orden secuencial.

El grafo de tareas busca identificar las **actividades a realizar (sus nodos)** y las **relaciones que hay entre las mismas (conexiones)**, con lo cual podemos ver cuáles son las tareas que necesitan terminar antes de que otras den inicio; el grafo de tareas nos ayuda a **identificar** de manera precisa la **secuencia de ejecución** de los procesos, así como los puntos donde el problema es paralelizable o no. La principal utilidad que se le puede dar a estos grafos es el organizar de mejor manera la ejecución de actividades antes de asignarlas a un hilo.

Relacionado con el grafo de tareas tenemos la **ruta crítica**, que para este caso hace referencia a la secuencia de tareas dependientes, las cuales determinan el **tiempo mínimo** que se requiere **para completar una tarea**, visto desde el punto de vista del grafo, sería la **ruta más larga** considerando la **duración de cada una de las tareas** ejecutadas, por lo que representa el límite del rendimiento total. Es necesario conocer esta ruta ya que, aunque el problema se pueda paralelizar en varios puntos, hay ciertas tareas que dependen de otras, y no pueden evitarse agregando más hilos de ejecución.

La **ruta crítica** tiene un papel importante en el desarrollo de un procedimiento paralelo, ya que **nos deja ver** dónde realmente **falla el sistema**, dónde hay **cuellos de botella**, ya que si hay una tarea que forma parte de esta ruta, entonces su retraso sería perjudicial directamente al rendimiento, incrementando el tiempo de ejecución del sistema total, mientras que las actividades que no forman parte de la ruta pueden tener retrasos menores sin afectar directamente a la ejecución total del programa. Esto nos deja claro que para buscar un **buen rendimiento** de una aplicación paralela **no es necesario paralelizar cada una de las tareas**, sino identificar cuáles de estas **actividades son críticas** y determinan en su conjunto, el tiempo total de ejecución.

Para ejemplificar de mejor manera la utilidad del grafo de tareas y de la ruta crítica, se tiene una **aplicación**, la cual **recibe una gran cantidad de datos**, y esta aplicación no los recibe únicamente, igual los **valida, luego los procesa y finalmente proporciona una salida**. Hay tareas secundarias que podemos paralelizar, como registrar otros eventos o preparar alguna estructura de apoyo, pero la base **no puede ser paralela**, ya que para procesar un dato, primero se necesita validarlo, y para dar una salida se necesita procesar primero, por lo cual **estas actividades** son las que **forman la ruta de dependencia** que buscamos, y por lo tanto son las que determinan en mayor medida el tiempo de ejecución del programa.

3. Granularidad de Tareas

La granularidad en un proceso paralelo, se refiere al **componete mínimo** con el que **el sistema puede ejecutarse** de forma paralela, o sea, el trabajo que es asignado a cada tarea en una ejecución paralela. Esto es importante ya que gracias a la granularidad podemos repartir el **trabajo de manera más uniforme** a cada uno de los hilos, logrando así que el rendimiento del sistema total no se vea afectado por una mala división de tareas. Existen diversos tipos de granularidad, los cuales son:

- **Granularidad gruesa**: en este tipo de granularidad se requiere de dividir un problema en pocas tareas, haciendo que cada una de estas sea más grande y realice varias operaciones en su ejecución,

de igual forma cada una de estas tareas tiene un tiempo de ejecución grande; aquí no hay comunicación tan constante entre tareas.

- **Granularidad fina**: hace referencia a una división de tareas más pequeñas, es decir, un problema se divide en un número de procesos mayor, haciendo que cada uno de estos realice un trabajo más detallado; el hecho de que las tareas sean más pequeñas requiere que exista una comunicación más constante entre las mismas.
- **Granularidad media**: la granularidad media, como su nombre lo indica, es un punto medio entre la granularidad fina y gruesa, no divide en varias tareas pequeñas, pero tampoco en varias tareas grandes, toma un punto intermedio donde divide el problema en una cantidad de tareas adecuado, las cuales a su vez toman un tiempo de ejecución adecuado, esto nos ayuda a balancear la carga sobre cada uno de los procesos ejecutados.

El **uso** de cada uno de estos tipos de granularidad **depende mucho de qué algoritmo e incluso a qué hardware** se quiere aplicar, y la necesidad que haya para resolver el problema. En el caso de elegir una **granularidad fina**, el beneficio es que se **aprovecha todo el paralelismo** disponible, pero por otro lado, **incrementa el costo de la administración** de estas tareas, ya que el sistema requiere crear, sincronizar, y coordinar varios procesos al mismo tiempo. En el caso de la **granularidad gruesa**, nos ahorramos el problema del sobrecarga administrativa, sin embargo, **limita la utilidad del proceso paralelo**, ya que dificulta dividir el problema en tareas más uniformes. Por último, tenemos la granularidad media, parecería la mejor opción por ser un punto medio entre la gruesa y la fina, lo cual nos evitaría ambos problemas, sin embargo, no siempre es así, hay algoritmos, sistemas, problemas, etc, donde la granularidad fina o gruesa hacen un muy buen trabajo, evitando la necesidad de ocupar una media.

4. Thread pools (pool de hilos)

Los **Thread pools** son un grupo de **hilos reutilizables** para ejecutar tareas concurrentes de manera eficiente, haciendo un uso menor de recursos. Cuando se deciden hacer **múltiples tareas** se pueden crear varios hilos y destruirlos cada que ya no se requieran, entonces, en lugar de hacer eso para no gastar tanto, este grupo de hilos se mantiene y se puede reutilizar, lo que reduce la **latencia** (haciendo las tareas más rápido) y aumenta la **escalabilidad** (mejora de rendimiento) lo que disminuye la sobrecarga de creación de subprocesos y **optimiza el uso del CPU**.
Entonces, destruir y crear hilos cada vez que se requiera, no es una idea muy factible porque cada que se crean hilos se hace una llamada al sistema operativo, el cual asigna y actualiza estructuras de datos para cada hilo, lo cual consume demasiados ciclos de procesamiento, que se podrían aprovechar mejor para otras cosas. Además se puede sobrecargar, llenándolo de demasiadas tareas que le sean difíciles de completar. También, como limita el número de subprocesos, evita que se agoten los recursos causando un error.
Por lo que el uso de Thread pools ayuda a gestionar tareas que se deben desarrollar de manera **simultánea** asignando un tiempo equitativo entre cada tarea. Es decir, al iniciar el sistema se asigna un número fijo de subprocesos los cuales esperan la realización de tareas paralelas se mantienen en reposo cuando no existe algún trabajo que se deba de hacer. Los grupos de subprocesos definen el **límite**, fijan un número de subprocesos que se encuentran activos al mismo tiempo, lo que evita el agotamiento de los recursos de memoria, tiempo del CPU, recursos del sistema, lo que reduce el tiempo que el CPU pierde alternando entre hilos, es decir, cambiando de contexto.
Una de las cosas no tan favorables de utilizar los Thread pools es que al englobar múltiples tareas en un **grupo de hilos** es más difícil encontrar los problemas que se presentan, algunos subprocesos se pueden **bloquear** cuando algunas de las tareas tardan más de lo debido, lo que afecta a otras tareas, lo mismo cuando sucede una **sincronización** que no es adecuada, se pueden causar interbloqueos. Además que se necesita ajustar el tamaño del grupo para lograr determinar y equilibrar el rendimiento para utilizar los recursos de manera adecuada.
Por lo tanto, cuando se reutilizan los hilos para gestionar las tareas concurrentes se reduce la sobrecarga,

aumenta la eficiencia del rendimiento y los recursos se administran controladamente.
Se puede decir que los thread pools son una manera de implementar el paralelismo de una manera eficiente, ya que permite ejecutar varias tareas a la vez, donde estas tareas se ejecutan en threads, con los thread pools definidos desde el inicio, se pueden asignar tareas desde una cola, decidiendo quién ejecuta logrando eficiencia, escalabilidad y estabilidad.

5. Sobrecosto de Paralelización (Overhead)

El paralelizar puede resultar **muy beneficioso para mejorar el rendimiento** de un programa, como se planteará más adelante con la ley de Amdahl, por lo que cuando se quieren realizar múltiples **tareas simultáneas** se puede requerir el uso de **múltiples threads** que las realicen puede resultar en un **overhead** si no se gestionan adecuadamente, lo que puede afectar directamente el **speedup** y la eficiencia causando que el programa paralelo posea un rendimiento menor al de su versión secuencial, como se vio anteriormente una manera de que no suceda esto es eficientar la creación y destrucción de hilos es el uso de **thread pools**.

Se puede decir que, así como el paralelismo mejora el rendimiento de los programas, también el usarlo puede llevar a problemas que no suelen presentarse en problemas de forma secuencial. Algunas veces puede parecer que al paralelizar todo el programa podríamos eficientar o mejorar el tiempo de ejecución, muchas veces esto no es así, aunque preferentemente la parte secuencial debería ser poca para mejorar el tiempo de ejecución de acuerdo con la **ley de Amdahl** (explicada en el siguiente apartado), hay partes del código que aunque se paralelicen no aumentan su velocidad significativamente, esto para bucles en paralelo que tienen muy pocas iteraciones.
Cuando se tiene el acceso a la **memoria compartida** entre varios hilos se puede tener overhead, esto porque en los **códigos secuenciales** se crean **variables estáticas** sin problema, pero en un entorno paralelo el acceso concurrente a ellas puede generar condiciones de carrera, para evitarlo, se requieren implementar mecanismos de sincronización como son los locks, que garantizan una exclusión mutua, pero usarlo crea un sobrecosto significativo debido a que los threads pueden quedar bloqueados en lo que esperan los recursos compartidos reduciendo la efectividad del paralelismo, por ello es mejor evitar o disminuir el uso de memoria compartida sobre todo en bucles cuando hay paralelización.
Otro de los problemas que limita los beneficios del paralelismo es el **hardware disponible**, porque si el número de hilos sobrepasa la cantidad de procesadores disponibles entonces estos tendrán que compartir el CPU, lo que aumenta el cambio de contexto y reduce por ende, su rendimiento. Por eso, muchas veces es mejor paralelizar únicamente el bucle exterior, guardando la paralelización del bucle interno para cosas específicas, como cuando existen operaciones con alto sobrecosto o cuando el hardware es suficiente para soportarlo. Por lo que el paralelismo debe aplicarse de manera estratégica y no indiscriminadamente para no introducir un overhead considerable que disminuya el rendimiento en lugar de mejorarlo.

6. Ley de Amdahl y Speedup

La **ley de Amdahl** puede determinar hasta qué punto se puede acelerar un programa al paralelizarlo, es decir, qué tanto puede mejorar su rendimiento al optimizar una parte de él. Para ello, debemos conocer lo implica la **mejora de rendimiento** o en su término en inglés **'speedup'** realiza un aumento lineal con el número de unidades de procesamiento que tenga el sistema cuando no se toma en cuenta la naturaleza de dicho programa, es decir, cuando se considera un programa en un modelo ideal. La naturaleza del programa es fundamental conocerla para saber la medida del speedup, por eso se debe considerar el problema, lo que busca solucionar, el tiempo que tarda en solucionarlo y su manera de solucionarlo.
En los programas paralelizados existen partes del código que no se pueden paralelizar, estas partes son las partes del código secuencial (ejecutado por un elemento del procesador). Las cuales no se pueden

paralelizar por requerimientos del problema, por razones de dependencia o que sólo tienen accesos a recursos únicos, lo que limita la mejora de rendimiento, a pesar del aumento de los núcleos. Por lo que la mejora de rendimiento dependerá de:

- Cuánto tiempo tarda en ejecutarse la parte del código serie.
- Cuánto tiempo tarda en ejecutarse la parte del código paralelizable
- El número de operaciones realizadas en forma paralela en el tiempo de ejecución del código paralelizable.

Así es como se definió la ley de Amdahl en 1967 por Gene Amdahl que indicó que el rendimiento de un programa dependerá del número de procesadores máximo que se aprovecharán para la ejecución del programa de manera simultánea. Si un programa contiene más partes de código paralelizado aumentará el límite de mejora de rendimiento, mientras que si la mayoría es código secuencial disminuirá su máximo de mejora de rendimiento, incluso aunque se aumente el número de núcleos.

Por lo anterior, es que la Ley de Amdahl dicta que:

$$Speedup_n = \frac{T_1}{T_n} = \frac{1}{\frac{F_p}{n} + Fs} = \frac{1}{\frac{F_p}{n} + (1 - F_p)} \tag{1}$$

Donde, T_1 es el tiempo de ejecución en un núcleo (sin paralelismo), T_n es el tiempo de ejecución en n núcleos, F_p es la parte del programa que sí se puede paralelizar, mientras que $(1 - F_p)$ es la parte que no se puede paralelizar.
Cuando se agregan más núcleos n se acelera la parte del código en paralelo, mientras que la parte secuencial no cambia. Aunque $n \to \infty$ si un porcentaje del código es secuencial su máximo de speedup está limitado a esa parte secuencial.
Entonces, al aumentar el número de núcleos, la parte secuencial no desaparece, en cambio, se vuelve más dominante, lo que disminuye es el tiempo necesario para la parte paralela del código del programa. Esto se muestra en la Figura 1.

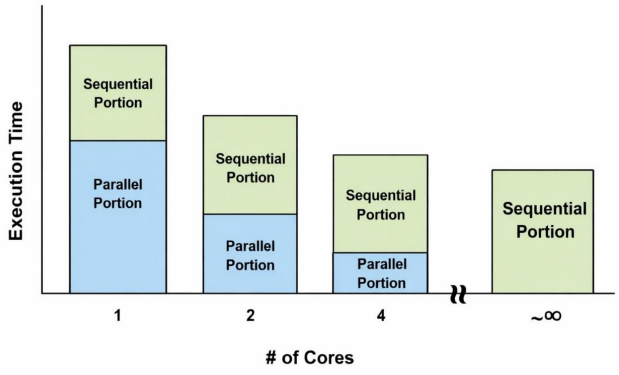

Figura 1: Ley de Amdahl.

7. Ley de Gustafson

La ley de Gustafson tiene que ver con el escalamiento débil, al contrario de la Ley de Amdahl que es de escalamiento fuerte.
La **Ley de Gustafson** es un modelo que describe el rendimiento de los sistemas paralelos desde una perspectiva optimista. A diferencia de otros enfoques, no asume que el tamaño del problema sea fijo, sino que plantea que este puede crecer conforme se agregan más procesadores. En este sentido, la parte secuencial del programa (aquella que no se puede paralelizar) se mantiene constante, mientras que la parte paralela aumenta proporcionalmente al número de procesadores. Esto permite que el sistema realice más trabajo en el mismo tiempo de ejecución, logrando una aceleración cercana a la lineal en la mayoría de las veces.

Por otro lado, **la Ley de Amdahl** opta por una visión más conservadora. Este modelo supone que el tamaño del problema es fijo y que solo se busca reducir el tiempo de ejecución mediante la paralelización. Es por ello que la mejora en el rendimiento está limitada por la parte secuencial del programa, la cual no puede beneficiarse del uso de múltiples procesadores. Como resultado, incluso con un número elevado de procesadores, la ganancia de velocidad tiene un límite, lo que restringe la eficiencia de la paralelización.

Se puede decir que la **principal diferencia** entre ambas leyes radica en la forma en que consideran el **tamaño del problema** y el **uso de los recursos**. Mientras la ley de Amdahl evalúa la mejora en tiempo para un problema fijo (escalabilidad fuerte), la ley de Gustafson propone aumentar la carga de trabajo conforme crecen los recursos disponibles (escalabilidad débil). Por ello, la primera es útil para identificar límites teóricos de aceleración, mientras que la segunda resulta más representativa de **escenarios reales** donde los sistemas aprovechan su capacidad adicional para resolver problemas más grandes. La ley de Gustafson puede verse de la siguiente manera:

$$S = (1 - f) + (f \times N) \tag{2}$$

Donde S es el Speedup, f es la parte del programa que sí se puede paralelizar y N número se procesadores.

En cuanto a su aplicación, la ley de Gustafson es adecuada cuando el objetivo no es únicamente acelerar una tarea, sino ampliar su alcance aprovechando recursos adicionales. Esto implica que, al aumentar el tamaño del problema, la porción paralelizable del trabajo tiende a dominar, mientras que la parte secuencial permanece relativamente pequeña, lo que favorece una eficiencia elevada incluso con muchos **procesadores o GPU**. Por esta razón, en campos como el modelado climático o el entrenamiento de modelos de gran escala, es común observar mejoras casi lineales en el rendimiento total. En esencia, esta ley respalda la idea de que incrementar la capacidad de cómputo no solo reduce tiempos, sino que habilita la resolución de problemas significativamente más grandes dentro del mismo periodo de ejecución.

Conclusión

La planificación de tareas y la ejecución paralela son elementos clave en los sistemas operativos, ya que permiten aprovechar de manera eficiente los recursos computacionales y mejorar el rendimiento de los programas. A través de conceptos como el paralelismo de datos y de tareas, los grafos de dependencias, la granularidad de tareas y el uso de thread pools, es posible organizar y distribuir el trabajo de forma estratégica. No obstante, el paralelismo no siempre garantiza mejores resultados, ya que factores como el overhead, las dependencias entre tareas y las limitaciones del hardware pueden afectar negativamente el desempeño al paralelizar. En este sentido, la Ley de Amdahl demuestra que el speedup está limitado por la parte secuencial del programa, lo que resalta la importancia de identificar qué partes realmente conviene paralelizar y cuáles no. Sin embargo, la Ley de Gustafson ofrece una perspectiva complementaria al señalar que, en lugar de mantener fijo el tamaño del problema, es posible escalar la carga de trabajo conforme aumentan los recursos disponibles, logrando así un mejor aprovechamiento del paralelismo. Por lo que podemos concluir que el paralelismo debe aplicarse de manera estratégica, no solo optimizando

el tiempo de ejecución, sino también considerando la posibilidad de resolver problemas más grandes de forma eficiente, evitando sobrecostos innecesarios.

Bibliografía

[1] Pure Storage. (s. f.). *¿Qué es el paralelismo de datos?*. https://www.purestorage.com/la/knowledge/what-is-data-parallelism.html

[2] ScienceDirect. (s. f.). *Task parallelism*. https://www.sciencedirect.com/topics/computer-science/task-parallelism

[3] Microsoft. (s. f.). *Paralelismo de datos (biblioteca paralela de tareas)*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/parallel-programming/data-parallelism-task-parallel-library

[4] GeeksforGeeks. (2025, July 15). *Precedence graph in operating system*. https://www.geeksforgeeks.org/operating-systems/precedence-graph-in-operating-system/

[5] WebProgramacion Consultoría Informática. (s. f.). *Concurrencia de procesos*. https://webprogramacion.com/concurrencia-de-procesos/

[6] Team Asana. (2026, February 14). *Cómo utilizar el método de la ruta crítica en la gestión de proyectos*. Asana. https://asana.com/es/resources/critical-path-method

[7] Levy, F. K., Thompson, G. L., & Wiest, J. D. (1963, September). *The ABCs of the critical path method*. Harvard Business Review. https://hbr.org/1963/09/the-abcs-of-the-critical-path-method

[8] Mikel. (2004, September 6). *2.2 Paralelismo*. En *El manual para el clustering con openMosix* (Versión 1.0). LuCAS. https://ibiblio.org/pub/linux/docs/LuCaS/Manuales-LuCAS/doc-manual-openMosix-1.0/doc-manual-openMosix_html-1.0/node9.html

[9] ScienceDirect. (s. f.). *Grained parallelism*. https://www.sciencedirect.com/topics/computer-science/grained-parallelism

[10] Foster, I. (1995). *1.3 A parallel programming model*. En *Designing and Building Parallel Programs*. Argonne National Laboratory. https://www.mcs.anl.gov/~itf/dbpp/text/node9.html

[11] *Problemas potenciales en el paralelismo de datos y tareas - .NET.* (s.f.). Microsoft Learn: Build with answers in reach. https://learn.microsoft.com/es-es/dotnet/standard/parallel-programming/potential-pitfalls-in-data-and-task-parallelism

[12] Whitfield, B. (2025, 24 de junio). *What is Amdahl's Law? (Definition, Formula, Examples) | Built In*. Built In. https://builtin.com/articles/amdahls-law

[13] 3.5.1 Amdahl's Law (Sun Studio 12 Update 1: C User's Guide). (s.f.). *Oracle*. https://docs.oracle.com/cd/E19205-01/820-7598/bjaem/index.html

[14] Speedups and Amdahl's Law. (2024, 14 de marzo). *College of Engineering|Oregon State University.* [Archivo PDF]. https://web.engr.oregonstate.edu/~mjb/cs575/Handouts/speedups.and.amdahls.law.6pp.pdf

[15] McCool, M., Robison, A. D., Reinders, J. (2012). *Background*. En M. McCool, A. D. Robison, J. Reinders (Eds.), *Structured parallel programming* (pp. 39-75). Morgan Kaufmann. https://doi.org/10.1016/B978-0-12-415993-8.00002-5.

[16] *Parallelism.* (s.f.). Physically Based Rendering: From Theory to Implementation. https://www.pbr-book.org/4ed/Utilities/Parallelism#:~:text=If%20two%20threads%20modify%20the,with%20a%20value%20of%20two:

[17] Casini, D. Biondi, A. Buttazzo, G. (2019). *Análisis de tareas paralelas en tiempo real implementadas con grupos de subprocesos*. En *Proceedings of the 56th ACM/IEEE Design Automation Conference(DAC)* (pp. 1-6).IEEE.

[18] Bittla, S. (24 de mayo de 2025). *Leyes de rendimiento de las GPU: cuándo usar Amdahl, Gustafson, Moore y más.* Medium. `https://bittla.medium.com/gpu-laws-of-performance-when-to-use-amdahl-gustafson-moore-and-more-93e08fe9679b`